第2次練習題目-隨堂-QZ2

學號：_____

姓名：_____

作業撰寫時間：_____ mins（請填實際時間，包含程式撰寫時間）

最後撰寫文件日期：2026/10/29

本份文件包含以下主題：

[x] 說明內容

[x] 其他

說明內容

本次練習使用 Unity 的 Input System 偵測鍵盤輸入。程式放在 Assets/Scripts/PlayerController.cs，類別名稱為 PlayerController。腳本需要附加到場景中啟用的 GameObject，Unity 才會在播放時呼叫它的 Update()。

1. 偵測 W 鍵並輸出訊息

先在檔案開頭匯入 UnityEngine 與 UnityEngine.InputSystem。在 Update() 中，以 Keyboard.current 取得目前的鍵盤。若沒有鍵盤，提早結束這次 Update()，避免讀取不存在的物件。

W 鍵的判斷區塊如下：

```
if (keyboard.wKey.isPressed)
{
    Debug.Log("玩家按了 W 鍵！");
}
```

keyboard.wKey 代表鍵盤上的 W 鍵，isPressed 表示目前是否處於按住狀態。條件成立時，Debug.Log() 將指定訊息輸出到 Unity Console。這個寫法依照 Topic 8 第 8.7 節的範例使用 isPressed，所以持續按住 W 時，每次執行 Update() 都會再輸出訊息。

2. 在同一份檔案加入 A、S、D 鍵

A、S、D 的偵測方式相同，分別改用 aKey、sKey、dKey。四個條件使用各自獨立的 if，因此每次 Update() 都會檢查四個按鍵。若同時按住不同按鍵，符合的條件都可以輸出訊息。

PlayerController.cs 完整程式碼如下：

```
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerController : MonoBehaviour
{
    void Update()
    {
        Keyboard keyboard = Keyboard.current;

        if (keyboard == null)
        {
            return;
        }

        if (keyboard.wKey.isPressed)
        {
            Debug.Log("玩家按了 W 鍵！");
        }

        if (keyboard.aKey.isPressed)
        {
            Debug.Log("玩家按了 A 鍵！");
        }

        if (keyboard.sKey.isPressed)
        {
            Debug.Log("玩家按了 S 鍵！");
        }

        if (keyboard.dKey.isPressed)
        {
            Debug.Log("玩家按了 D 鍵！");
        }
    }
}
```

每列程式碼的意義

以下行號對應上面的完整程式碼，包含空白行。

行號	程式碼	說明
1	using UnityEngine;	匯入 Unity 引擎的命名空間，讓程式可以使用 MonoBehaviour 與 Debug 等類別。
2	using UnityEngine.InputSystem;	匯入 Input System 的命名空間，讓程式可以使用 Keyboard 類別。
3	空白行	分隔命名空間匯入與類別宣告，不影響執行。
4	public class PlayerController : MonoBehaviour	宣告公開的 PlayerController 類別，並繼承 MonoBehaviour，使它能作為 Unity 物件上的腳本元件。
5	{	開始 PlayerController 類別的內容。

行號	程式碼	說明
6	void Update()	宣告不回傳資料的 Update 方法。啟用的元件在遊戲執行時，Unity 會每幀呼叫此方法。
7	{	開始 Update() 的方法內容。
8	Keyboard keyboard = Keyboard.current;	宣告 Keyboard 型別的區域變數 keyboard，將目前鍵盤的參照存入其中。
9	空白行	分隔取得鍵盤與檢查鍵盤的程式，不影響執行。
10	if (keyboard == null)	檢查目前是否沒有可用的鍵盤。== 是相等比較，null 表示沒有物件參照。
11	{	開始沒有鍵盤時要執行的區塊。
12	return;	結束這一次 Update() 的呼叫，略過後面的按鍵判斷；它不會結束整個遊戲。
13	}	結束沒有鍵盤時的判斷區塊。
14	空白行	分隔防護檢查與 W 鍵判斷，不影響執行。
15	if (keyboard.wKey.isPressed)	檢查 W 鍵現在是否被按住。
16	{	開始 W 鍵條件成立時要執行的區塊。
17	Debug.Log("玩家按了 W 鍵！");	將「玩家按了 W 鍵！」輸出到 Console。雙引號內是字串，分號表示這個敘述結束。
18	}	結束 W 鍵的判斷區塊。
19	空白行	分隔 W 與 A 鍵的判斷，不影響執行。
20	if (keyboard.aKey.isPressed)	檢查 A 鍵現在是否被按住。
21	{	開始 A 鍵條件成立時要執行的區塊。
22	Debug.Log("玩家按了 A 鍵！");	將「玩家按了 A 鍵！」輸出到 Console。
23	}	結束 A 鍵的判斷區塊。
24	空白行	分隔 A 與 S 鍵的判斷，不影響執行。
25	if (keyboard.sKey.isPressed)	檢查 S 鍵現在是否被按住。
26	{	開始 S 鍵條件成立時要執行的區塊。
27	Debug.Log("玩家按了 S 鍵！");	將「玩家按了 S 鍵！」輸出到 Console。
28	}	結束 S 鍵的判斷區塊。
29	空白行	分隔 S 與 D 鍵的判斷，不影響執行。
30	if (keyboard.dKey.isPressed)	檢查 D 鍵現在是否被按住。
31	{	開始 D 鍵條件成立時要執行的區塊。

行號	程式碼	說明
32	Debug.Log("玩家按了 D 鍵！");	將「玩家按了 D 鍵！」輸出到 Console。
33	}	結束 D 鍵的判斷區塊。
34	}	結束 Update() 方法。
35	}	結束 PlayerController 類別。

Unity 設定與測試步驟

1. 在 Package Manager 確認已安裝 Input System。這個專案使用版本 1.20.1。

2. 在 Project Settings 的 Player 設定確認 Active Input Handling 使用 Input System Package (New)。

3. 在場景建立 Player 物件，將 PlayerController.cs 附加到該物件上。

4. 儲存腳本與場景，等待 Unity 編譯完成，確認 Console 沒有編譯錯誤。

5. 開啟 Console 視窗並按下 Play。

6. 點擊 Game 視窗，讓鍵盤輸入送到遊戲，再分別按住 W、A、S、D。

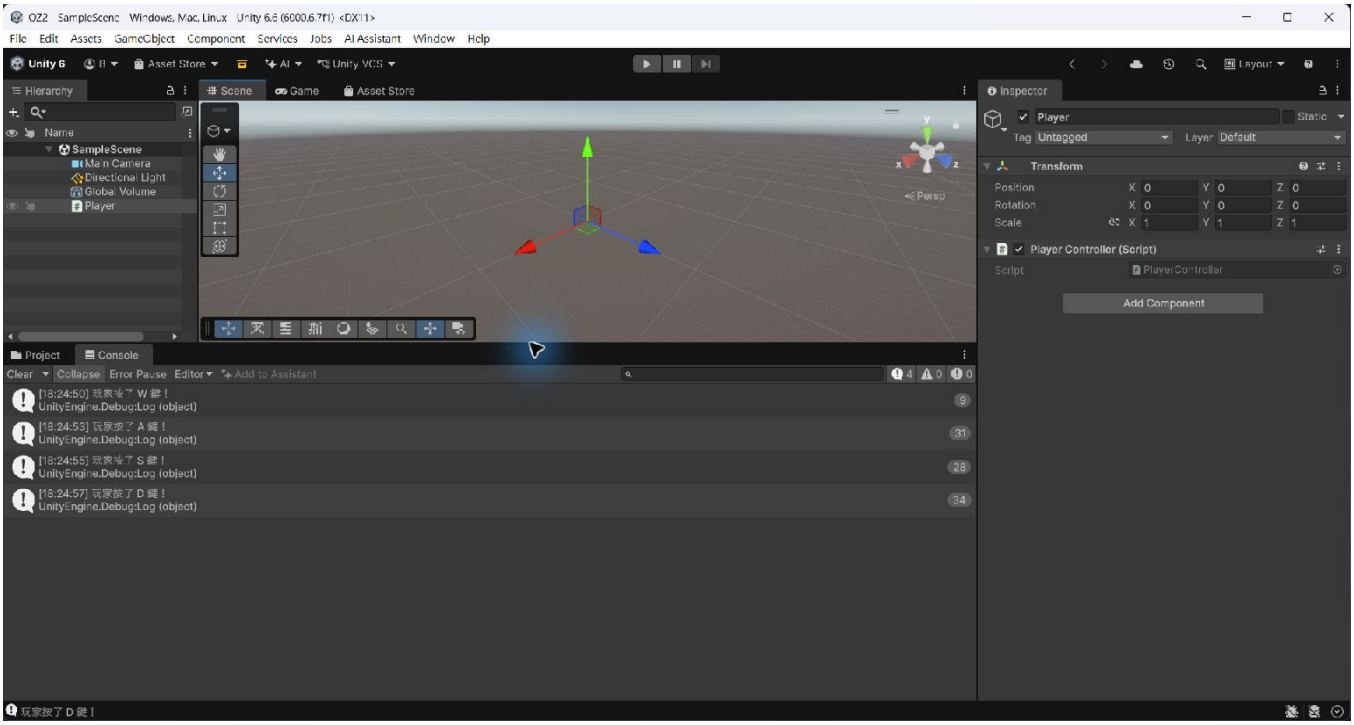
7. 確認 Console 顯示對應文字，並截圖保存程式碼、腳本掛載狀態與執行結果。

執行結果

本次在 Unity Play Mode 中，以 Input System 模擬按鍵的按下與放開事件進行測試。測試前後的遊戲影格由 518 增加到 556，確認遊戲有持續更新；四種按鍵均產生下表訊息。測試完成後已停止播放。

操作	預期 Console 訊息	實測狀態
按住 W	玩家按了 W 鍵！	通過
按住 A	玩家按了 A 鍵！	通過
按住 S	玩家按了 S 鍵！	通過
按住 D	玩家按了 D 鍵！	通過

執行畫面與腳本元件



Console 訊息局部放大（同一張原始截圖）

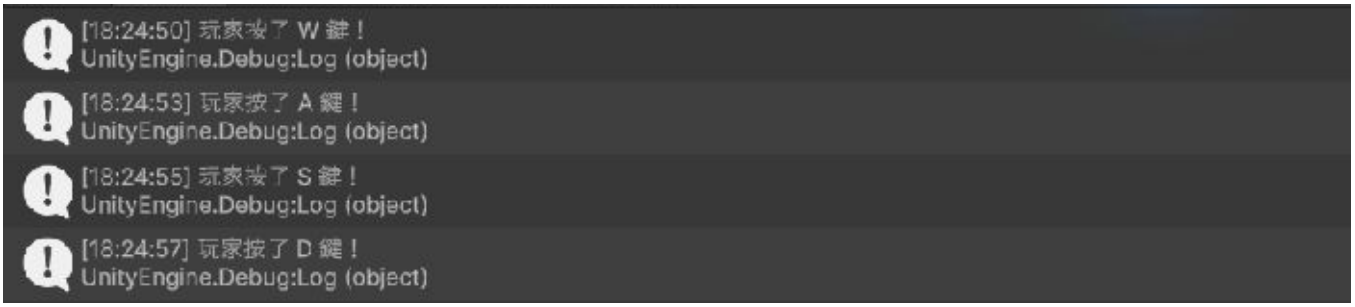


圖 1：左側 Hierarchy 的 Player 已附加右側 Inspector 顯示的 Player Controller 元件。下方 Console 顯示 W、A、S、D 的四種訊息，並且警告與錯誤數量皆為 0。此圖同時呈現第 1 題 W 鍵與第 2 題 A、S、D 鍵的執行結果。

Console 已開啟 Collapse，會將內容相同的輸出合併顯示。右側 9、31、28、34 是這次測試各訊息的累計次數；持續按住期間會反覆輸出，不同次數取決於按住時間與更新次數。

其他

Update() 每一幀都會執行，isPressed 則在按住期間持續成立。因此一次較長的按鍵操作可能產生多筆相同訊息，這符合本次練習採用的偵測方式。

Keyboard.current 可能為 null，先檢查再讀取按鍵可以避免 NullReferenceException。此外，腳本附加到啟用的 GameObject、遊戲進入 Play Mode，以及 Game 視窗取得輸入焦點，都是看到正確結果需要檢查的項目。